Appendix

PEAS

Task Environment Properties
Fully observable (vs. partially observable) An agent's sensors give it access to the complete state of the environment at each point in time. **Deterministic (vs. stochastic)** The next state of the environment is completely determined by the current state and the action executed by the agent. (If the environment is deterministic except for the actions of other agents, then the environment is **strategic**) **Episodic (vs. sequential)** The agent's experience is divided into atomic "episodes" (each episode consists of the agent perceiving and then performing a single action), and the choice of action in each episode depends only on the episode itself. **Static (vs. dynamic)** The environment is unchanged while an agent is deliberating. (The environment is **semi-dynamic** if the environment itself does not change with the passage of time, but the agent's performance score does) **Discrete (vs. continuous)** A limited number of distinct, clearly defined percepts and actions. **Single agent (vs. multi-agent)** An agent operating by itself in an environment.

Uninformed and Informed Search

Tree Search

```
create frontier
insert initial state
while frontier is not empty:
    state = frontier.pop()
    for action in actions(state):
        next state = transition(state, action)
        if next state is goal: return solution
        frontier.add(next state)
return failure
```

Graph Search

```
create frontier
create visited
insert initial state to frontier
while frontier is not empty:
    state = frontier.pop()
    if state is goal: return solution
    if state in visited: continue
    visited.add(state)
    for action in actions(state):
        next state = transition(state, action)
        frontier.add(next state)
return failure
```

Admissibility

A heuristic h(n) is admissible if for every node n, $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to reach the goal state from n.

Consistency

A heuristic h(n) is consistent if for every node n, every successor n' of n generated by any action a, $h(n) \leq c(n,a,n') + h(n')$ and $h(G) = 0$.

Consistency implies admissibility

Dominance

If $h2(n) \geq h1(n)$ for all n, then h2 dominates h1.

Appendix

Local Search

Hill Climbing

```
current = initial state
loop:
    neighbor = a highest value successor of current
    if value(neighbor) <= value(current):
        return current
    current = neighbor
```

Appendix

Adversarial Search

Minimax

```
def minimax(state):
    v = max_value(state)
    return action in successors(state) with value v

def max_value(state):
    if is_terminal(state): return utility(state)
    v = -∞
    for action, next_state in successors(state):
        v = max(v, min_value(next_state))
    return v

def min_value(state):
    if is_terminal(state): return utility(state)
    v = ∞
    for action, next_state in successors(state):
        v = min(v, max_value(next_state))
    return v
```

Alpha-beta Pruning

```
def alpha_beta_search(state):
    v = max_value(state, -∞, ∞)
    return action in successors(state) with value v

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for action, next_state in successors(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for action, next_state in successors(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```

Decision Trees

Decision Tree Learning

```
def DTL(examples, attributes, default):
    if examples is empty: return default
    if examples have the same classification:
        return classification
    if attributes is empty:
        return mode(examples)
    best = choose_attribute(attributes, examples)
    tree = a new decision tree with root best
    for each value v_i of best:
        examples_i = {rows in examples with best = v_i}
        subtree = DTL(examples_i, attributes – best, mode(examples))
        add a branch to tree with label v_i and subtree subtree
```

Entropy

For a data set containing p positive and n negative examples:

$$I(\frac{p}{p+n}, \frac{n}{p+n}) = -\frac{p}{p+n}log_2\frac{p}{p+n} - \frac{n}{p+n}log_2\frac{n}{p+n}$$

Information Gain

A chosen attribute A divides the training set E into subsets $E_1, \ldots, E_n$ according to their values for A, where A has v distinct values.

$$IG(A) = I(\frac{p}{p+n}, \frac{n}{p+n}) - remainder(A)$$

$$remainder(A) = \sum_{i=1}^{v} \frac{p_i + n_i}{p+n} I(\frac{p_i}{p_i+n_i}, \frac{n_i}{p_i+n_i})$$

Performance Measure

Confusion Matrix

For a classification problem with two labels, TRUE and FALSE, we can construct a confusion matrix as follows.

		Actual Label	
		TRUE	FALSE
Predicted Label	TRUE	True Positive (TP)	False Positive (FP)
	FALSE	False Negative (FN)	True Negative (TN)

Precision

$P = TP/(TP + FP)$

Recall

$R = TP/(TP + FN)$

F1 Score

$F1 = 2/(1/P + 1/R)$

Math

Logarithm
$log_a(x) = log_b(x)/log_b(a)$ for any base b